
Algorithm 1 World Action Planner (WAP)

Require: VLM Agent, low-dimensional robot controller ϕ , world-model WM, (Optional) policy π ,

Input: Task description ℓ , initial state s , environment env,

while not done **do**

▷ **Agent Action Proposal**

Propose primitive actions using the VLM agent, and then translate into robot actions using controller: $g = \text{Agent.ProposeActions}(s, \ell)$, $a = \phi(g, s_{\text{proprio}})$

▷ **Global Optimization Guided by Agent Feedback**

Imagine the actions using the world model: $\hat{s}_{\text{next}} = \text{WM}(s, a)$

Optimize based on agent feedback: $\Delta g = \text{Agent.Optimize}(\hat{s}_{\text{next}}, \ell)$, $a = \phi(g + \Delta g, s_{\text{proprio}})$

▷ **Local Search with Agent Ranking**

Generate action candidates: $a_1, a_2, \dots, a_N = \text{GridSearch}(a)$

Imagine action candidates: $\hat{s}_i = \text{WM}(s, a_i)$, $i = 1, 2, \dots, N$

if use policy **then**

Imagine the policy rollout after the candidate action: $\hat{s}_i = \text{WM}(\hat{s}_i, \pi(\hat{s}_i))$, $i = 1, 2, \dots, N$

end if

Select the best one using VLM agent: $i^* = \text{argmax Agent.Rank}(\hat{s}_1, \hat{s}_2, \dots, \hat{s}_N, \ell)$

▷ **Execute the Actions**

Execute the actions after search: $s, \text{done} = \text{env.Step}(s, a_{i^*})$

if use policy **then**

Roll out the policy following the actions a_{i^*} : $s, \text{done} = \text{env.Step}(s, \pi(s))$

end if

end while
